Universidad Católica San Pablo

Curso de Interacción Humano–Computador

Avance final

Five Nights of Terror

Juego de vigilancia con interacción entre PC, tablet y cámara

Curso: Interacción Humano–Computador (CS2H1)
Estudiantes: Marcelo Torres y Anthony Briceño
Semestre: 2026-02
Universidad: Universidad Católica San Pablo

Arequipa, Perú · 2026

Índice

1. Carátula **1**

2. Descripción del aplicativo **2**

3. Problemas encontrados y soluciones planteadas **3**

4. Discusión sobre las opiniones de los individuos de prueba **4**

5. Conclusiones **5**

6. Referencias bibliográficas **6**

2. Descripción del aplicativo

Five Nights of Terror es un prototipo de juego de terror y vigilancia en el que la persona debe sobrevivir desde las 12:00 a. m. hasta las 6:00 a. m. durante seis noches. La experiencia reparte la atención entre un monitor de PC, donde se ve la escena tridimensional, y una tablet, donde se resuelven minitareas y se controla la caja musical. Una cámara web permite estimar si el jugador dirige la cabeza hacia la pantalla. Si un animatrónico lo alcanza, la noche termina en derrota; al sobrevivir hasta las 6:00 a. m. se desbloquea la siguiente, y completar la sexta produce la victoria final.[R1]

El juego propone tres presiones simultáneas. **Freddy** puede avanzar cuando el jugador deja de mirar el PC; **Vixy** se vuelve peligrosa cuando se acumulan tareas sin resolver; y **Puppet** depende del nivel de una caja musical que se vacía con el tiempo. Cada amenaza requiere una respuesta distinta: volver la vista al monitor, completar tareas o dar cuerda a la caja desde la tablet. Esta alternancia es la mecánica central, no un efecto secundario de la interfaz.

El prototipo integra componentes que antes podían ejecutarse de forma separada. En la versión examinada, Unity toma las decisiones sobre tiempo, tareas, movimiento y ataques; Flutter presenta controles y minijuegos; un relay en Python transmite mensajes entre ambos. El informe describe esa versión del proyecto. No presenta resultados de una evaluación formal con participantes, porque no consta una prueba completa de ese tipo en los materiales revisados.

- **Vigilancia con cámara.** MediaPipe Unity Plugin obtiene puntos faciales y el componente `HeadOrientationTracker` calcula una proporción entre frente, nariz y mentón. El estado booleano `mirandoPC` se usa para la mecánica de Freddy y para la prueba del tutorial.
- **Escena 3D y enemigos.** Unity 2022.3.30f1 muestra el entorno, mueve los animatrónicos mediante rutas y `NavMeshAgent`, controla la caja musical, reproduce señales sonoras y presenta los resultados de la partida.
- **Tablet interactiva.** Flutter implementa el menú, el tutorial, la lista de tareas y diez minijuegos: cables, perillas, secuencia, ritmo, WiFi, ventiladores, temperatura, procesamiento de datos, subida de datos y trazado de curso. La caja musical es una acción permanente aparte de esa lista.
- **Comunicación bidireccional.** Unity y Flutter intercambian mensajes JSON mediante WebSocket. `UnityRelay` los reenvía según el rol de cada conexión; el relay no ejecuta las reglas del juego.
- **Continuidad.** El progreso por noche se almacena en `PlayerPrefs`. Tras una derrota puede repetirse la noche; tras una victoria se habilita la siguiente hasta la sexta.



El flujo comienza con `connect`. Unity inicializa la noche y envía `task_list`, `night_status` y `estado_puppet`. La tablet presenta estos estados y, al terminar una tarea, responde con `task_completed` o `task_failed`. Unity actualiza el estado de los enemigos y, cuando corresponde, publica `attack` y `game_over`. La cámara se procesa en el PC; el intercambio entre dispositivos se basa en eventos y estados, no en transmitir el video de la webcam.

El uso de WebSocket permite que ambos extremos reciban eventos durante una sesión sin abrir una conexión nueva para cada interacción. El estándar define un canal bidireccional; en el

prototipo, el contenido de cada mensaje se expresa como JSON y la lógica de negocio permanece en Unity.

3. Problemas encontrados y soluciones planteadas

Durante la última actualización, dejando de lado un poco la parte mecánica, pasamos por las pruebas de los *testers*. Al leer sus comentarios, varios de ellos coincidían en los mismos problemas al usar nuestro juego, como los siguientes:

- **Problema con el script de Windows:** en la creación para la parte de iniciar el juego, creamos scripts para cargar tanto la conexión de la tablet como el inicio de Unity. Este funcionaba perfectamente al crear el `.ps1`, pero hubo problemas con Windows que no supimos identificar.
- **Respuesta táctil de Puppet:** hubo problemas con el envío de la señal hacia el servidor al presionar el reinicio de la cajita musical (figura 1). El gesto se cancelaba si el dedo se deslizaba unos píxeles y la cuerda se soltaba; ahora la caja musical y la subida de datos leen el toque de forma directa.
- **Tareas sin culminar:** cuando queríamos hacer las tareas de la tablet, estas se salían o regresaban al menú de tareas sin haberlas culminado, y nos aparecían como que no las habíamos hecho, lo que generaba problemas con la interacción del juego. Un claro ejemplo es la tarea de procesar datos: si hacíamos un falso *touch* en un lugar no indicado, se perdía el avance. Ahora un deslizamiento equivocado solo reinicia la secuencia y no termina la tarea.
- **No sabían quién los mató:** uno de los *testers* no lograba identificar qué animatrónico lo había atrapado. La pantalla de derrota ahora dice **TE ATRAPÓ** seguido del nombre, con su ícono y el motivo (figura 2): Freddy, por no mirar la pantalla; Vixy, por tener tareas pendientes; y Puppet, por dejar la caja musical sin cuerda.
- **Tablet difícil de leer y de tocar:** la flecha de volver y el botón de cerrar pasaron de 48 a 80 píxeles de lado, se respetó el área segura de la pantalla para que botones como *Enviar* no quedaran tapados, y se agrandaron los textos y los elementos de todos los minijuegos. Además se agregaron avisos por vibración.
- **Cables solo por color:** los cables se distinguían únicamente por su color; ahora cada uno tiene además una forma distinta (figura 1).
- **Las primeras noches eran muy fáciles:** Freddy toleraba entre 35 y 20 segundos sin ser mirado, según la noche, y ahora tolera entre 22 y 12; además, Vixy avanza cada 6 segundos en lugar de cada 8.
- **La tablet decía «Conectado» sin el juego:** el estado cambiaba a conectado al instante, aunque el juego estuviera cerrado. Ahora solo se muestra cuando Unity responde y, si no lo encuentra en 8 segundos, avisa «No se encontró el juego» y permite reintentar. También se corrigieron los textos del tutorial, que estaban escritos en voseo.



Figura 1: Izquierda: cajita musical con la interfaz cambiada y mejor respuesta táctil. Centro y derecha: cables solo por color y con formas

4. Discusión sobre las opiniones de los individuos de prueba

Este apartado se centrará en la discusión sobre las opiniones de mejora de nuestro juego que dieron los *testers*.

Algo en que coincidían muchos de los *testers* era que a veces, cuando interactuaban con la tablet para hacer las minitareas, estas no respondían. Podía ser un problema de que no presionaban en los lugares clave o simplemente que el servidor no recibía la señal del *touch*.

También hicieron énfasis en la parte del screamer del animatrónico, ya que sentían que era lo esencial en este estilo de juegos. Dicho esto, agregamos un video como parte final al recibir un screamer, como en la figura 2.

Los videos de Freddy y de Puppet fueron generados con una herramienta de inteligencia artificial generativa (Gemini)[R6], a partir de imágenes de los propios modelos del juego, y luego recortados al fragmento del salto. El de Vixy no usó inteligencia artificial: se elaboró con un programa propio que anima su imagen con acercamiento, temblor y destellos. En los tres casos el sonido es el del juego.

Algunos se enfocaron en las mecánicas y en cómo aprovecharlas. Por ejemplo, si la tablet se ponía a una altura en que la cámara pudiera detectar la frente o el mentón, figuraba como *true*; de este modo, Freddy, que depende de la cámara para atacar, no realizaba movimientos. Por eso planteamos que el videojuego se base en tener la tablet a un costado, para evitar este tipo de pérdidas de experiencia.

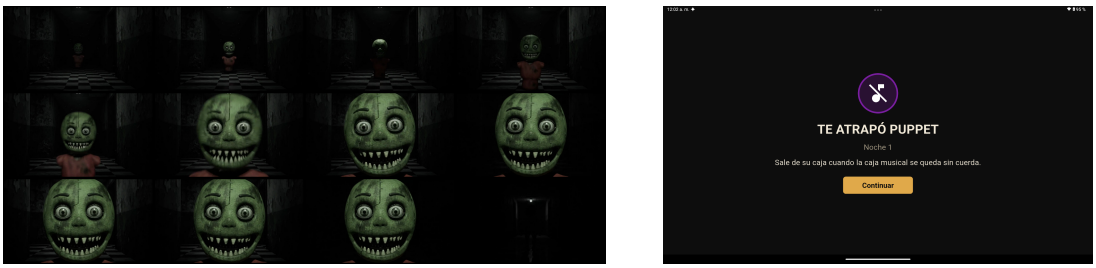


Figura 2: Izquierda: jumpscare de Freddy. Derecha: pantalla de derrota en la tablet, que indica qué animatrónico atrapó al jugador

5. Conclusiones

1. El prototipo logra integrar vigilancia en PC, minitareas en tablet y detección de rostro en una misma dinámica. Unity concentra las reglas y el relay mantiene el intercambio entre dispositivos, lo que permite describir con claridad el origen de cada evento.
2. El tutorial de tres pantallas transforma las instrucciones en una pequeña práctica: muestra la cámara real y permite probar una tarea antes de comenzar. Es una mejora de comprensión y preparación, aunque su eficacia debe comprobarse con jugadores nuevos.
3. La especificidad del sistema genera sus principales límites. La cámara no equivale siempre a intención de mirar; la alternancia de pantallas añade carga cognitiva, y algunos gestos temporizados o sostenidos no ofrecen todavía una alternativa. La corrección de cables mediante formas demuestra que una limitación puede mitigarse con cambios concretos de interfaz.
4. Las correcciones hechas a partir de los comentarios de los *testers* (respuesta táctil, identificación del animatrónico que ataca, dificultad y jumpscare) atienden los problemas observados, pero todavía deben validarse con nuevos jugadores para confirmar que resuelven la experiencia.
5. Como trabajo posterior se recomienda evaluar partidas completas con participantes, registrar pérdidas de detección y tiempos de tarea, probar distintas cámaras y condiciones de luz, y revisar señales redundantes y opciones de accesibilidad. Estas son recomendaciones de validación, no resultados ya medidos.

6. Referencias bibliográficas

[R1]
Flutter. (2026). *Building user interfaces with Flutter*. https://docs.flutter.dev/ui.

[R2]
MediaPipe Unity Plugin. (s. f.). *Repositorio del proyecto*. https://github.com/homuler/MediaPipeUnityPlugin.

[R3]
Unity Technologies. (2022). *NavMesh Agent, Unity Manual 2022.3*. https://docs.unity3d.com/2022.3/Documentation/Manual/class-NavMeshAgent.html.

[R4]
Fette, I. y Melnikov, A. (2011). *The WebSocket Protocol*. RFC 6455. Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc6455/.

[R5]
World Wide Web Consortium. (s. f.). *Understanding Success Criterion 1.4.1: Use of Color*. Web Content Accessibility Guidelines 2.2. https://www.w3.org/WAI/WCAG22/Understanding/use-of-color.html.

[R6]
Google. (2026). *Gemini*. https://gemini.google.com.